Projected Gradient Descent for Smooth Convex Optimization

1 Algorithm Description

The algorithm iteratively updates parameters by taking gradient steps and projecting the result back onto the constraint set at each iteration. It is designed for smooth convex optimization problems with closed, convex, and bounded constraint domains.

1.1 Mathematical Formulation

We are given a smooth convex optimization problem where we want to minimize a convex function $f : \mathbb{R}^n \to \mathbb{R}$ subject to the constraint that $\mathbf{w} \in \mathcal{C}$, where $\mathcal{C}$ is a closed, convex, and bounded set. The algorithm iteratively updates the parameter vector $\mathbf{w}$ using the following steps:

1. **Gradient Computation**: Calculate the gradient of the objective function at the current parameter vector:
$$\nabla f(\mathbf{w}_t)$$
2. **Gradient Step**: Update the parameters by taking a step in the direction of the negative gradient:
$$\mathbf{w}_{t+1/2} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t)$$
where $\eta > 0$ is the step size.
3. **Projection Step**: Project the updated parameters back onto the constraint set $\mathcal{C}$:
$$\mathbf{w}_{t+1} = \mathrm{Proj}_{\mathcal{C}}(\mathbf{w}_{t+1/2})$$
where $\mathrm{Proj}_{\mathcal{C}}(\mathbf{v}) = \arg\min_{\mathbf{u}\in\mathcal{C}} \|\mathbf{u} - \mathbf{v}\|_2$.

Hyperparameters:

- η (Step Size): Controls the magnitude of the update at each iteration.
- T (Number of Iterations): Determines how long the algorithm runs.

2 Pseudo Code and Dry Run

2.1 Pseudocode

```
Initialize: w_0   ^n (within constraint set C),   > 0, T > 0
for t = 0 to T-1 do
    Compute gradient: g_t =  f(w_t)
    Gradient step: w_t_half = w_t -   * g_t
    Projection step: w_(t+1) = Proj_C(w_t_half)
end for
Return: w_T
```

2.2 Dry Run Example

Consider the quadratic function $f(w) = (w - 3)^2$. We apply the algorithm with:

- Initialization: $w_0 = 0$
- Step size: $\eta = 0.1$
- Constraint set: $\mathcal{C} = \mathbb{R}$ (no actual projection needed since $\mathcal{C}$ is the entire real line)

2.2.1 Iteration Details

Iteration 0:

- Compute Gradient: $\nabla f(w_0) = 2(w_0 - 3) = 2(0 - 3) = -6$
- Gradient Step: $w_{0.5} = 0 - 0.1 \times (-6) = 0.6$
- Projection Step: $w_1 = w_{0.5} = 0.6$ (since $\mathcal{C} = \mathbb{R}$)
- Objective Value: $f(w_1) = (0.6 - 3)^2 = 5.76$

Iteration 1:

- Compute Gradient: $\nabla f(w_1) = 2(w_1 - 3) = 2(0.6 - 3) = -4.8$
- Gradient Step: $w_{1.5} = 0.6 - 0.1 \times (-4.8) = 1.08$
- Projection Step: $w_2 = w_{1.5} = 1.08$
- Objective Value: $f(w_2) = (1.08 - 3)^2 = 3.6864$

Iteration 2:

- Compute Gradient: $\nabla f(w_2) = 2(w_2 - 3) = 2(1.08 - 3) = -3.84$
- Gradient Step: $w_{2.5} = 1.08 - 0.1 \times (-3.84) = 1.464$
- Projection Step: $w_3 = w_{2.5} = 1.464$
- Objective Value: $f(w_3) = (1.464 - 3)^2 = 2.359296$

The objective function value is decreasing with each iteration, demonstrating the effectiveness of the projected gradient descent algorithm on this simple quadratic function.

3 Convergence Theory

3.1 Assumptions

We consider the optimization problem:

$$\min_{x \in \mathcal{X}} f(x)$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a smooth and convex function, and $\mathcal{X} \subseteq \mathbb{R}^n$ is a closed, convex, and bounded set. The algorithm iteratively updates parameters using the following rule:

$$x_{k+1} = \Pi_{\mathcal{X}}(x_k - \eta \nabla f(x_k))$$

where $\Pi_{\mathcal{X}}$ denotes the projection onto the set $\mathcal{X}$, and $\eta > 0$ is the step size.

3.1.1 Formal Statement of Assumptions

1. **Smoothness**: The function f is L-smooth, i.e., there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,
$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|.$$
2. **Convexity**: The function f is convex, meaning for all $x, y \in \mathcal{X}$ and $\theta \in [0, 1]$,
$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y).$$
3. **Constraint Set**: The domain $\mathcal{X}$ is closed, convex, and bounded.

3.2 Main Convergence Theorem

Under the assumptions stated above, the sequence $\{x_k\}$ generated by the projected gradient descent algorithm satisfies:

1. **Convergence to Optimal Solution**: The sequence converges to the optimal solution x^* of the problem.
2. **Rate of Convergence**: The rate of convergence is $O(1/k)$, where k is the iteration number. Specifically,

$$f(x_k) - f(x^*) \leq \frac{2L\|x_0 - x^*\|^2}{k}$$

for a suitable choice of step size $\eta \leq \frac{1}{L}$.

3.3 Theoretical Properties

3.3.1 Stability Analysis

The algorithm is stable for step sizes $0 < \eta \leq \frac{1}{L}$. If η exceeds $\frac{1}{L}$, the convergence may no longer be guaranteed.

3.3.2 Sensitivity to Hyperparameters

The choice of step size η is crucial. Setting η too small results in slow convergence, while too large a value could lead to divergence or oscillatory behavior.

3.3.3 Comparison to Baseline Methods

Compared to unprojected gradient descent, the projected variant ensures feasibility with respect to constraints at every iteration. This method is more suitable for constrained problems where maintaining feasibility is critical.

3.4 Discussion

The projected gradient descent algorithm provides a robust method for solving smooth convex optimization problems with constraints. The convergence properties outlined demonstrate its effectiveness and limitations. While the convergence rate of $O(1/k)$ is typical for first-order methods, improvements may be achievable through acceleration techniques or adaptive step size strategies. Future work could investigate these enhancements or explore extensions to non-convex scenarios.

4 Convergence Proof

4.1 Preliminaries (Definitions and Lemmas)

4.1.1 Definitions

- **Projection**: For a closed and convex set $\mathcal{X}$, the projection of a point y onto $\mathcal{X}$ is defined as:

$$\Pi_{\mathcal{X}}(y) = \arg\min_{x \in \mathcal{X}} \|x - y\|^2$$

- **L-smoothness**: A function f is L-smooth if:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^n$$

- **Convexity**: A function f is convex if:

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y) \quad \forall x, y \in \mathcal{X}, \theta \in [0,1]$$

4.1.2 Lemmas

[Projection Property] For any $x \in \mathbb{R}^n$ and $z \in \mathcal{X}$, the projection satisfies:

$$\langle x - \Pi_{\mathcal{X}}(x), z - \Pi_{\mathcal{X}}(x) \rangle \leq 0$$

[Descent Lemma] If f is L-smooth, then for any $x, y \in \mathbb{R}^n$:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2$$

4.2 Main Proof (Step-by-Step Derivation)

4.2.1 Algorithm Update Rule

The update rule for projected gradient descent is:

$$x_{k+1} = \Pi_{\mathcal{X}}(x_k - \eta \nabla f(x_k))$$

4.2.2 Step-by-Step Derivation

1. **Projection Property**:
$$\langle x_k - \eta \nabla f(x_k) - x_{k+1}, x^* - x_{k+1} \rangle \leq 0$$
Expanding and rearranging gives:
$$\langle x_k - x_{k+1}, x^* - x_{k+1} \rangle \leq \eta \langle \nabla f(x_k), x_{k+1} - x^* \rangle$$

2. **Descent Lemma Application**: Using the descent lemma:
$$f(x_{k+1}) \leq f(x_k) + \langle \nabla f(x_k), x_{k+1} - x_k \rangle + \frac{L}{2}\|x_{k+1} - x_k\|^2$$
Substitute the projection inequality:
$$f(x_{k+1}) \leq f(x_k) + \eta \langle \nabla f(x_k), x^* - x_{k+1} \rangle + \frac{L}{2}\|x_{k+1} - x_k\|^2$$

3. **Bounding the Gradient Term**: From convexity:
$$f(x_k) - f(x^*) \leq \langle \nabla f(x_k), x_k - x^* \rangle$$
Using the smoothness property:
$$\langle \nabla f(x_k), x_k - x^* \rangle \leq f(x_k) - f(x^*) + \frac{L}{2}\|x_k - x^*\|^2$$

4. **Combining the Inequalities**: Combine the above inequalities:
$$f(x_{k+1}) - f(x^*) \leq (1 - \eta)(f(x_k) - f(x^*)) + \frac{L\eta^2}{2}\|\nabla f(x_k)\|^2$$

5. **Telescoping Sum**: Sum over $k = 0$ to $K - 1$:
$$\sum_{k=0}^{K-1}(f(x_{k+1}) - f(x^*)) \leq \sum_{k=0}^{K-1}(1 - \eta)(f(x_k) - f(x^*)) + \frac{L\eta^2}{2}\sum_{k=0}^{K-1}\|\nabla f(x_k)\|^2$$

4.3 Rate Derivation (With Constants)

Assume $\eta \leq \frac{1}{L}$, then:

$$f(x_k) - f(x^*) \leq \frac{2L\|x_0 - x^*\|^2}{k}$$

This implies a convergence rate of $O(1/k)$.

4.4 Special Cases

- **Exact Line Search**: If exact line search is used, the step size η can be adjusted dynamically to achieve faster convergence.
- **Strong Convexity**: If f is strongly convex, the convergence rate improves to $O(1/k^2)$.